

Seminar2: VARIABLES & STRUCTURES in PL/SQL

1. Declaration and initialization process

- the declaration of variables is made in the declarative area (delimited by DECLARE) of the block (or sub-block);
- initialization can be done at the declaration or in the execution area (between BEGIN and END);
- the variables will be visible in the rest of the block, respectively also in the blocks included in it, except in the sub-blocks where their name is redefined (as in most structured programming languages, the meaning of a user-defined name in a block/sub-block is given by the closest declaration prior to the place of use);
- all PL/SQL variables have a data type, constraints, and a valid string of values;
- The declaration and initialization are carried out as follows:

Variable_name [CONSTANT] DATA_TYPE [NOT NULL] [:= | DEFAULT]

- the constants must be initialized, and subsequently they will not be able to change their value;
- the NOT NULL variables must be initialized, and later they will not be able to receive the NULL value;
- The following notation convention is used:

c_name Constant

v_name Variable

g_name Global variable (global variable defined in the program package specification area and valid for all subprograms of the package).

2. Types of variables

PL/SQL Variables	
	▪ Scalar ▪ Composed (Record) ▪ Reference (Cursor) ▪ LOB (Large Objects): NCLOB, CLOB, BLOB, BFILE ▪ Object
Non-PL/SQL variables: environment variables (BIND VARIABLES), host variables (HOST VARIABLES), indicator variables (INDICATOR VARIABLES)	

a) Scalar variable:

Scalar types contain simple values (a scalar variable can contain only one simple value at a time). The most used types are:

- **CHAR (lung_max)** - fixed length of max 32,767 bytes
- **VARCHAR2 (lung_max)** – variable length of max 32,767 bytes
- **LONG** [2GB variable-length string]
- **NUMBER (precision, scale)** – fixed point, maximum precision 38 decimal places
- **BOOLEAN** (can take TRUE, FALSE, or NULL values) – does not have an equivalent SQL type
- **DATA**
- **BINARY_integer** and **PLS_INTEGER** (integers between -2147483647 and 2147483647) – are PL/SQL specific, use machine arithmetic so that the operations are faster compared to the NUMBER type
- **BINARY_FLOAT** and **BINARY_DOUBLE** (for real numbers, single and double precision, respectively)
- **TIMESTAMP (f)** – calendar date and time with precision f for milliseconds. There is also the **TIMESTAMP WITH TIME ZONE** variant as well as **TIMESTAMP WITH LOCAL TIME ZONE**

Examples of defining variables and constants:

```
v_funcție varchar2(9);  
v_numar binary_integer:=0;  
v_totalsal number(9,2):=0;  
v_datainceput date:=sysdate+7;  
c_taxa constant number(3,2):=8.25;  
v_valid boolean not null:=true;
```

Data types can have **subtypes**, either Oracle or user-defined.

Subtypes can introduce restrictions to the base type, containing a subset of it. There are also subtypes that do not introduce any restrictions to the base type, they can have a role in ensuring compatibility with other databases and programming languages.

For example, NUMBER has among its subtypes, DECIMAL for fixed point with a maximum precision of 38 decimal places and FLOAT for floating point with a precision of about 38 decimal places (it introduces practically no restriction to the basic type).

Type PLS_INTEGER has among its subtypes NATURAL for non-negative values and SIGNTYPE which can only take the values -1, 0, or 1. VARCHAR and STRING are subtypes with the same value range as VARCHAR2.

Example:

```
declare
m decimal; -- echivalent m NUMBER(38,0)
n float; -- echivalent n NUMBER
begin
m:=100.20;
dbms_output.put_line('m='||m);
n:=100.20;
dbms_output.put_line('n='||n);
End;
/
```

The execution of the block will display:

```
m=100
n=100.2
```

Example:

```
declare
z signtype;
begin
z:=2;
dbms_output.put_line('z='||z);
End;
/
```

Executing the block will throw an exception because the z variable can only take the values -1, 0, or 1.

PL/SQL variables are displayed via the PUT_LINE function in the **DBMS_OUTPUT** package. You can use the concatenation operator (||) to display multiple messages or variables on the same line.

DBMS_OUTPUT.PUT_LINE ('THE VALUE OF THE VARIABLE IS:' ||variable);

Populate variables with values from database tables

- Use the SELECT command with the INTO clause to populate PL/SQL variables with attribute values from tables;
- SELECT requests within PL/SQL blocks must provide only one result line (otherwise an error is reported).

Example:

```
--the employee's name is displayed with code 100
SET SERVEROUTPUT ON
DECLARE
v_last_name VARCHAR2(20);
BEGIN
SELECT last_name INTO v_last_name FROM employees WHERE employee_id =
100;
DBMS_OUTPUT.PUT_LINE('EMPLOYEE NAME:' || v_last_name);
END;
```

3. %TYPE attribute

Assigns a variable the type of another variable or the specific data type of a column in the table. It contributes to ensuring the independence of programs from the database.

Declare a variable with %TYPE:

Variable	table_name.column_name%TYPE;
or	
variable1	data_type;
variable2	variable1%TYPE;

Example:

```
-- Find the first and the last name for the customer that has the id 222.  
DECLARE  
First_name customers.cust_first_name%type;  
Last_name customers.cust_last_name%type;  
BEGIN  
Select cust_first_name, cust_last_name into first_name, last_name from customers  
where customer_id=222;  
Dbms_output.put_line('Client's name and surname: '||first_name||' '||last_name);  
END;
```

Note: The NOT NULL constraint of a column does not apply to the variable declared by using the %TYPE attribute.

4. Environment Variables or Link Variables of Host Applications (BIND VARIABLES)

- are variables related to the application in which the PL/SQL engine runs;
- must be declared in the application (in the host environment) and can be accessed and modified within PL/SQL blocks;
- after the execution of the PL/SQL block is finished, the variable remains in the host environment with the value received after running the block (and can be passed to another block, thus transmitting values between the PL/SQL blocks);
- may not be used in procedures, functions or packages;
- is declared outside the PL/SQL block using the VARIABLE keyword:

VARIABLE g_variable_name TYPE

- for declaring a host variable of type NUMBER, the precision and scale are not specified;
- for their use within a PL/SQL block or in an SQL phrase outside the block is prefixed with ":"

:host_variable:=v_variable;

- is displayed outside the block with the help of the PRINT command (when displaying the variable will not be prefixed with ":")

PRINT g_variable_name

Examples:

```
SET SERVEROUTPUT ON
VARIABLE n number

BEGIN
select count(*) into :n
from orders
where order = 'online';
END;
/
PRINT n
-- Question: What's the result?
begin
:n:=:n+5;
dbms_output.put_line('n'||:n);
-- N=?
:n:=:n+3;
dbms_output.put_line('n'||:n);
-- N=?
end;
/
PRINT n
-- Question: What's the result?
```

Note: You can auto-display the variable by setting **AUTOPRINT ON** (no need to use the PRINT command after executing the block).

Example:

```
--select the products and their price for those products that have the price < the
average price of the product with the code 3133 without using a nested select

SET SERVEROUTPUT ON
SET AUTOPRINT ON
VARIABLE g_price number

BEGIN
select avg(unit_price) into :g_price
from order_items
where product_id = 3133;
END;
/
select * from order_items where unit_price< :g_price;
```

5. Substitution variables

- as a rule, substitution variables are used to pass values from the SQL*Plus environment to SQL commands or PL/SQL blocks, while bind *variables* are used to pass values in reverse or to

transfer values between successively launched PL/SQL blocks (the first block sets the variable, the next queries it);

- through substitution variables, values can be passed to SQL commands or PL/SQL blocks launched (using "&" or "&&");
- can be invoked from SQL commands or PL/SQL blocks via "&nume_variabila" or "&&nume_variabila";
- are local to the SQL session in which they were declared;
- in the SQL environment, substitution variables can be easily read by entering values from the keyboard (using ACCEPT), can be defined (with DEFINE) or displayed on the screen (with PROMPT);

Example:

```
--the number of orders of the employee whose code is entered by the user through the
substitution variable is displayed id_angajat
DECLARE
v_no_orders number(2);
BEGIN
select count(order_id) into v_no_orders from orders
where sales_rep_id=&sales_rep_id;
dbms_output.put_line('The employee has: '|| v_no_orders||' orders');
END;
/
```

Note: In a PL/SQL block you can use all types of variables, but respecting their characteristics and rules of use. The following example uses both the substitution variable *s_first_nume* defined and initialized by the DEFINE command, as well as the *g_salary* link variable, as well as the local variable *v_first_name* of the same type as the name column in the Employees table. The substitution variable containing a string will be used with a right apostrophe in assignments or comparisons, e.g. name:='&v_nume'.

```
-- the employee's salary and first name are displayed with the name Abel
SET SERVEROUTPUT ON
VARIABLE g_salary number
DEFINE s_last_name=Abel
DECLARE
v_first_name employees.first_name%type;
BEGIN
select first name, salary into v_first_name, :g_salary
from employees where last_name='&s_last_name';
DBMS_OUTPUT.PUT_LINE ('The employee first name is: '||v_first_name);
END;
/
print g_salary
```

PROPOSED EXERCISES AND SOLVED EXERCISES

Specify what will be displayed when the following PL/SQL block is run:

```
DECLARE
v_var1 NUMBER :=100;
v_var2 NUMBER;
v_var3 NUMBER := v_var2;
V_Var4 VARCHAR2(20) := 'Variabila PL/SQL';
v_var5 NUMBER NOT NULL := v_var1;
c_const1 CONSTANT DATE := TO_DATE('03/03/2025','dd/mm/yyyy');
c_const2 CONSTANT NUMBER NOT NULL := 2;
c_const3 CONSTANT NUMBER := NULL;
v_var6 NUMBER DEFAULT NULL;

BEGIN
DBMS_OUTPUT.PUT_LINE('variable 1 = '||v_var1);
DBMS_OUTPUT.PUT_LINE('variable 2 = '||v_var2);
DBMS_OUTPUT.PUT_LINE('variable 3 = '||v_var3);
DBMS_OUTPUT.PUT_LINE('variable 4 = '||v_var4);
DBMS_OUTPUT.PUT_LINE('variable 5 = '||v_var5);
DBMS_OUTPUT.PUT_LINE('constant 1 = '||c_const1);
DBMS_OUTPUT.PUT_LINE('constant 2 = '||c_const2);
DBMS_OUTPUT.PUT_LINE('constant 3 = '||c_const3);
DBMS_OUTPUT.PUT_LINE('variable 6 = '||v_var6);
END;
/
```

Specify what will be displayed when the next PL/SQL block (which contains nested blocks, illustrating the scope of visibility of variables with the same name):

```
DECLARE
var NUMBER;
BEGIN
var := 1;
DBMS_OUTPUT.PUT_LINE(var);

    <<bloc1>>
    DECLARE
    var NUMBER;
    BEGIN
    var :=2;
    DBMS_OUTPUT.PUT_LINE(var);
    END bloc1;

DBMS_OUTPUT.PUT_LINE(was);

    <<bloc2>>
    DECLARE
    var NUMBER;
    BEGIN
    var :=3;
    DBMS_OUTPUT.PUT_LINE(var);
```

```
<<bloc3>>  
DECLARE  
var NUMBER;  
BEGIN  
var :=4;  
DBMS_OUTPUT.PUT_LINE(var);  
DBMS_OUTPUT.PUT_LINE(bloc2.var);  
END bloc3;  
DBMS_OUTPUT.PUT_LINE(var);  
END bloc2;  
DBMS_OUTPUT.PUT_LINE(var);  
END;  
/
```